

3주차 진단평가

1. [1 marks each]

Colours are represented by a computer as a combination of the three primary colours: red, green and blue.

Numerical values are used to represent the different shades of each primary colour. These values range from 0 to 255 in decimal, or 00 to FF in hexadecimal.

- (a) State why hexadecimal numbers are frequently used in computing.
- (b) State the number of bits used to represent a non-primary colour, such as yellow.
- (c) State the maximum number of colours that can be represented in a computer pixel.

2.

A laptop computer supplements its primary memory by making use of virtual memory.

- (a) Outline the use of paging in relation to virtual memory. [2]

The laptop has 1GB of random access memory (RAM) and a single processor. The laptop is using one of the latest operating systems to run multimedia gaming programs.

- (b) Explain the limitations and consequences of using the laptop for this purpose. [3]

One of the laptop's game applications stores the data relating to the different actions of one of its characters in a stack.

- (c) Suggest **one** reason why the character's actions might be stored in a stack. [2]

At specific moments during the game this data is read from the stack (S) into a queue (Q).

- (d) Using appropriate access methods for stacks and queues, construct an algorithm that reads the data from the stack and enters it into the queue. You should assume that the queue structure exists and that both structures are of a fixed size. [6]
- (e) Outline **one** advantage of making the queue dynamic. [2]

3.

- Images in computers are stored as two-dimensional arrays.

A black-and-white image (**Figure 1**) is stored as a 10×10 two-dimensional array named MAT (**Figure 2**).

Each element of MAT holds a number for a colour; 1 represents the colour black and 0 represents the colour white.

Figure 1: The simple black-and-white image

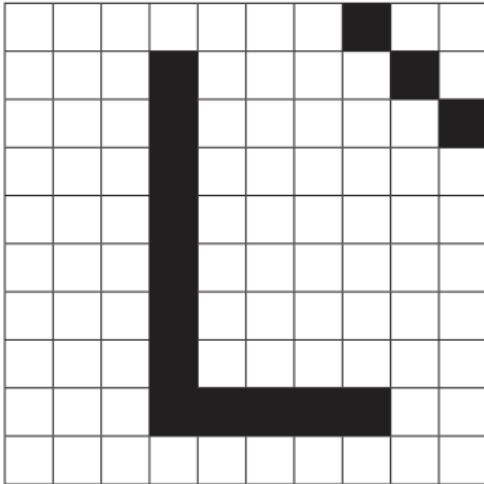


Figure 2: The corresponding two-dimensional array MAT

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
[0]	0	0	0	0	0	0	0	1	0	0
[1]	0	0	0	1	0	0	0	0	1	0
[2]	0	0	0	1	0	0	0	0	0	1
[3]	0	0	0	1	0	0	0	0	0	0
[4]	0	0	0	1	0	0	0	0	0	0
[5]	0	0	0	1	0	0	0	0	0	0
[6]	0	0	0	1	0	0	0	0	0	0
[7]	0	0	0	1	0	0	0	0	0	0
[8]	0	0	0	1	1	1	1	1	0	0
[9]	0	0	0	0	0	0	0	0	0	0

In an application, the black-and-white image can be inverted (all white pixels are changed to black, and all black pixels are changed to white).

Method `invert(N, A)` accepts a positive integer N and an $N \times N$ two-dimensional array A that holds the data for a simple black-and-white image; it returns the inverted $N \times N$ two-dimensional array A .

- (a) Construct an algorithm in pseudocode for the method `invert(N, A)`.

[3]

In the application, it is also possible to rotate an image clockwise by 90 degrees (90°). For example, when the simple black-and-white image is rotated, the corresponding 10×10 two-dimensional array MAT is updated.

This would mean the first row of the original MAT is the last column in the rotated MAT, the second row is the second-to-last column, ... and the last row is the first column.

Figure 3: The simple black-and-white image rotated by 90° (clockwise)

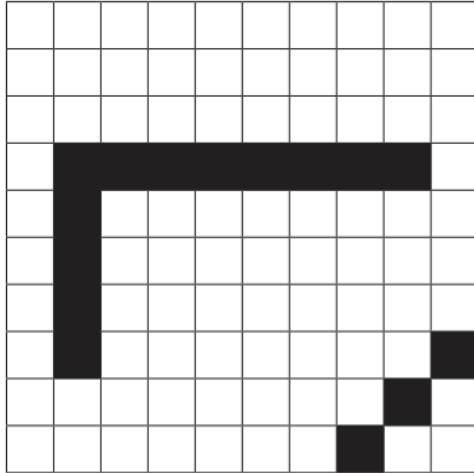


Figure 4: The corresponding two-dimensional array MAT

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
[0]	0	0	0	0	0	0	0	0	0	0
[1]	0	0	0	0	0	0	0	0	0	0
[2]	0	0	0	0	0	0	0	0	0	0
[3]	0	1	1	1	1	1	1	1	1	0
[4]	0	1	0	0	0	0	0	0	0	0
[5]	0	1	0	0	0	0	0	0	0	0
[6]	0	1	0	0	0	0	0	0	0	0
[7]	0	1	0	0	0	0	0	0	0	1
[8]	0	0	0	0	0	0	0	0	1	0
[9]	0	0	0	0	0	0	0	1	0	0

Consider the following algorithm fragment:

```

K=input()
loop for M=0 to K mod 4 - 1
    A=rotate(N,A)
end loop

```

where:

- N is an integer and A is the $N \times N$ two-dimensional array that holds data about an image
- $K (K \geq 0)$ is an integer showing how many times the image should be rotated
- method `rotate(N,A)` accepts an integer N and an $N \times N$ two-dimensional array A representing an image. It returns an $N \times N$ two-dimensional array representing the image rotated clockwise by 90° .

(b) (i) State the number of degrees by which the image will be rotated if the input value of K is 3. [1]

(ii) Outline why it is more efficient that the loop in the given algorithm fragment executes $(K \bmod 4)$ times instead of K times. You may give an appropriate example in your answer. [2]

The algorithm for method `rotate(N, A)` uses an additional $N \times N$ two-dimensional array, named `B`. The $N \times N$ dimensional array `B` is initialized and updated using the values from `A` to represent the image rotated clockwise by 90° .

(c) Construct the algorithm in pseudocode for the method `rotate(N, A)` described above. [3]

To avoid inefficient use of memory, a new algorithm for the method `rotate(N, A)` is constructed.

The $N \times N$ two-dimensional array `A` should be rotated clockwise by 90° , **without** the use of any additional arrays.

One way of rotating the two-dimensional array `A` clockwise by 90° is to transpose `A`, and then reverse each row of `A`.

The transpose of `A` is formed by turning all the rows of `A` into columns. For example, the value in the first row and third column (`A[1][3]`) is swapped with the value in the third row and first column (`A[3][1]`).

(d) Construct the new algorithm in pseudocode for the method `rotate(N, A)` described above. [6]